

Seguridad básica: usuarios en memoria y HTTP Basic

2. Seguridad básica: usuarios en memoria y HTTP Basic

¿Por qué empezar por algo que vas a sustituir?

Lo que ves aquí —usuarios en memoria y HTTP Basic— no es la versión final: dentro de dos apartados no quedará ni rastro de ninguna de las dos piezas. Si empezaras directamente por la versión final, tendrías código que funciona, pero no sabrías responder a "¿por qué hace falta proteger la contraseña?" — esas respuestas solo se entienden habiendo visto antes el problema real.

El camino completo: usuarios en memoria y HTTP Basic (aquí) → BCrypt en base de datos (próximo apartado) → JWT (dos apartados más adelante).

Lo primero que hace Spring Security: cerrarlo todo

Spring Security es el framework de seguridad de Spring: comprueba quién hace cada petición y qué tiene permiso para hacer. Se instala como una dependencia más de Maven, pero con una diferencia importante:

En cuanto está en el proyecto, empieza a actuar sola, con un comportamiento por defecto muy estricto, sin que hayas escrito ni una línea de configuración propia todavía.

Antes y después de spring-boot-starter-security

```
curl -i http://localhost:8080/api/v1/libros
```

Momento	Respuesta
Antes de la dependencia	200 OK, con el catálogo completo.
Con Spring Security en el proyecto	401 Unauthorized, sin cuerpo útil, con WWW-Authenticate: Basic realm="Realm".

Toda la API deja de responder libremente de golpe — cada petición exige autenticarse, incluida la que hace un momento era pública.

Mínima exposición, llevada al extremo

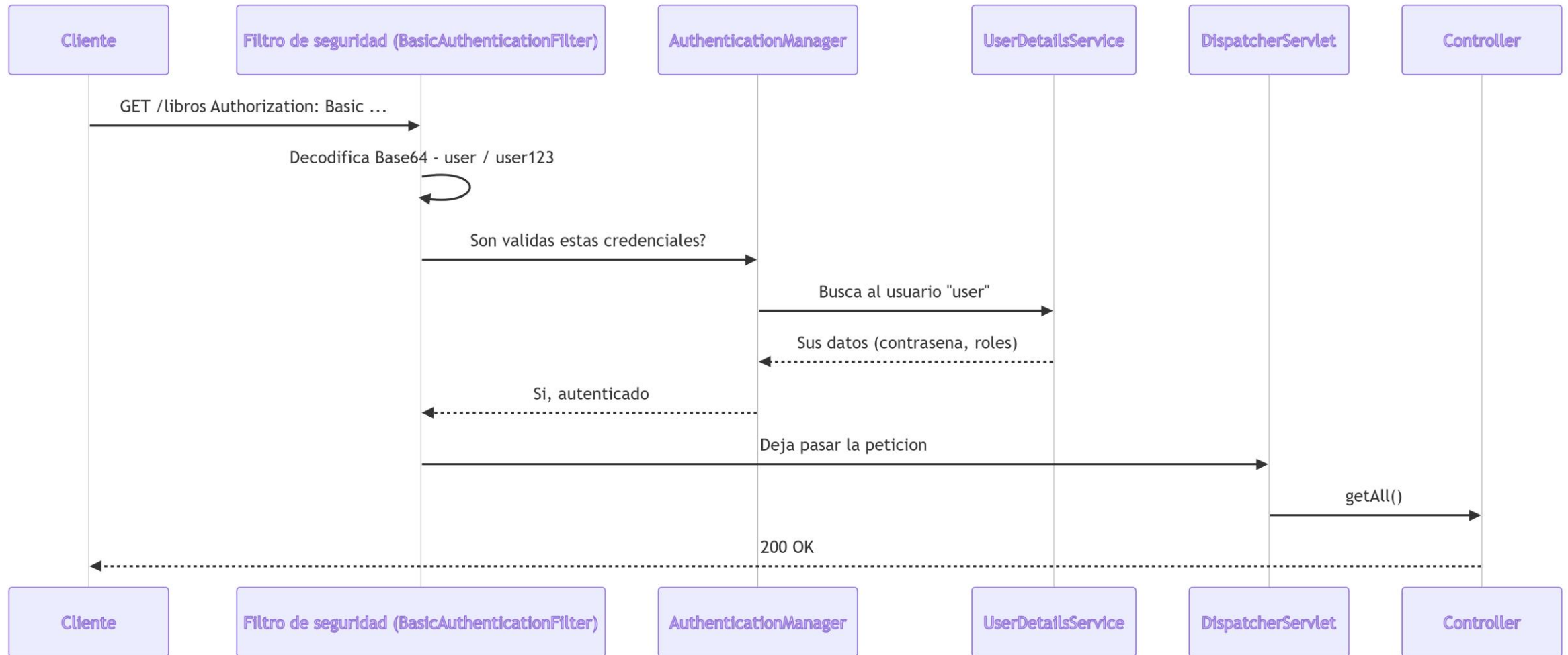
Es el principio de "mínima exposición" del apartado anterior, llevado al extremo por defecto: Spring Security asume que, si no se le dice explícitamente qué dejar abierto, todo debe estar cerrado.

Eso lo consigue insertando un filtro delante de todos los controllers: una pieza que intercepta la petición HTTP antes de que llegue al código de la aplicación, la inspecciona, y decide si la deja continuar o la corta ahí mismo. Spring Security no añade uno, sino una cadena entera de ellos (la filter chain).

Dónde encaja Spring Security en una petición

Antes de tocar código, conviene ver el mapa completo: por dónde pasa una petición desde que sale del cliente hasta que llega al Controller, y qué pieza decide, en cada paso, si la deja continuar. Mejor verlo en tres pasos, de menos a más complicado.

1. Cuando todo va bien



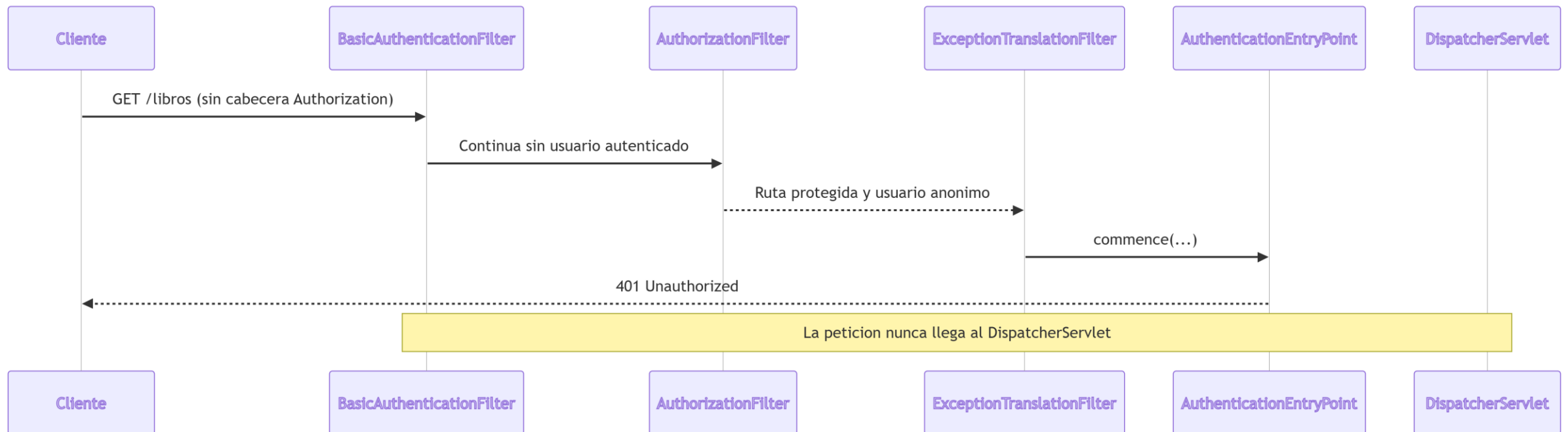
El filtro, el AuthenticationManager y el DispatcherServlet

El filtro intercepta la petición y decodifica la cabecera; el AuthenticationManager coordina la comprobación; el UserDetailsService es quien de verdad sabe si ese usuario existe y con qué contraseña y roles.

Si todo eso sale bien, la petición llega al DispatcherServlet — la pieza de Spring MVC que decide a qué método de qué controller mandar cada una. Es el mismo mecanismo de siempre: lo único que cambia es que, antes de llegar hasta él, la petición ahora tiene que pasar primero el control del filtro.

2. Sin ninguna cabecera Authorization

El fallo más simple: la petición no trae credenciales de ningún tipo.

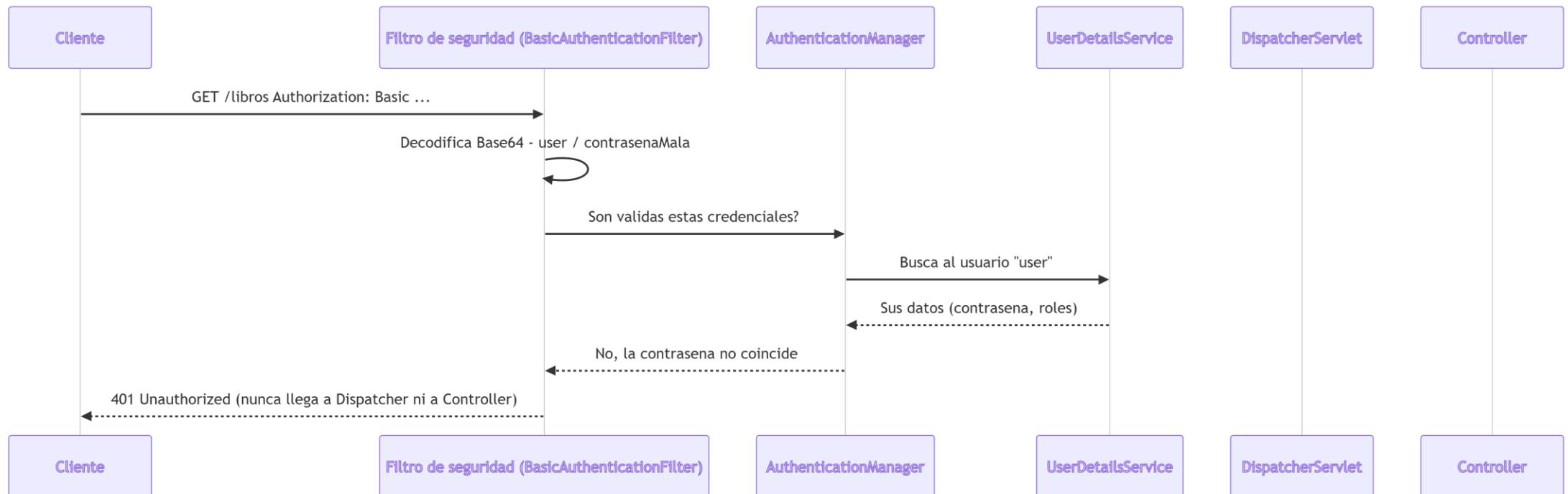


Ni `AuthenticationManager` ni `UserDetailsService` intervienen

`BasicAuthenticationFilter` continúa sin autenticar al usuario; después, la autorización detecta que un usuario anónimo intenta acceder a una ruta protegida y `ExceptionTranslationFilter` activa el `AuthenticationEntryPoint`, que devuelve el 401. Todo ocurre antes de llegar al `DispatcherServlet`.

3. Credenciales incorrectas

El caso menos evidente: la petición sí trae una cabecera Authorization con forma correcta, pero el usuario no existe o la contraseña no coincide.



Mismo circuito, distinto final

Este caso es casi idéntico al primero hasta el penúltimo paso — el filtro sí pregunta, AuthenticationManager sí consulta al UserDetailsService — y solo al final cambia la respuesta. El resultado es el mismo 401 que sin credenciales, pero el camino es bastante más largo.

Queda pendiente un tercer tipo de fallo: el 403

Los tres diagramas son sobre autenticación (¿quién eres?). Pero hay un cuarto caso: credenciales válidas de un usuario real que no tiene permiso para esa ruta en concreto — eso ya es autorización, y da un 403 Forbidden, no un 401. Lo trabajarás más adelante en el tema.

Autenticación vs. autorización

	Pregunta que responde	Ejemplo
Autenticación	¿Quién eres?	Comprobar que el usuario y la contraseña son correctos.
Autorización	¿Puedes hacer esto?	Comprobar que tienes permiso para borrar un libro del catálogo.

Puedes estar autenticado y aun así no autorizado para una acción concreta: son dos capas distintas.

La tarjeta de acceso de un edificio

Cuando presentas la tarjeta, el lector comprueba quién eres y que la tarjeta sigue activa: eso es autenticación. Después, la puerta concreta comprueba si esa identidad tiene permiso para abrirla: eso es autorización.

En Spring Security, `httpBasic(...)` configura el mecanismo de autenticación y `authorizeHttpRequests(...)` decide qué puede hacer el usuario autenticado en cada ruta.

HTTP Basic: cómo viaja usuario y contraseña

El cliente manda usuario y contraseña en una cabecera Authorization, codificados en Base64 — un sistema que convierte bytes en texto imprimible:

```
"user:user123" → codificar en Base64 → "dXNlcjpw1c2VyMTIz"
```

```
Authorization: Basic dXNlcjpw1c2VyMTIz
```

Base64 NO es cifrado

⚠ Cualquiera que intercepte la cabecera lee la contraseña

Si decodificaras esa cabecera —echo "dXNlcjp1c2VyMTIz" | base64 -d— obtendrías user:user123, en texto legible. Base64 es solo una codificación, no una forma de ocultar información. HTTP Basic solo es aceptable sobre una conexión cifrada con HTTPS — sin eso, la contraseña viaja prácticamente en claro en cada petición.

Usuarios en memoria: InMemoryUserDetailsManager

Es la pieza que en el diagrama de antes aparecía como UserDetailsService: quien responde "¿quién es el usuario user, y con qué contraseña y roles?"

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withUsername("user")
        .password("{noop}user123").roles("USER").build();

    UserDetails admin = User.withUsername("admin")
        .password("{noop}admin123").roles("ADMIN").build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

{noop} y el prefijo ROLE_

El prefijo {noop} le dice a Spring Security "esta contraseña está almacenada en texto claro y no se le aplica ningún PasswordEncoder; compárala tal cual".

💡 **.roles("USER") no es literalmente el rol**

Por debajo, Spring Security le añade el prefijo ROLE_: el rol real que queda registrado es ROLE_USER, no USER. No tienes que escribir ese prefijo tú mismo, pero si lo ves en un log o en un test, ya sabes de dónde sale.

Una primera política de acceso

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/v1/libros/**").permitAll()
            .anyRequest().authenticated()
        )
        .csrf(AbstractHttpConfigurer::disable)
        .httpBasic(Customizer.withDefaults())
        .build();
}
```

Léela como una lista de reglas, de arriba abajo

Pieza	Qué significa
<code>.requestMatchers(GET, "/api/v1/libros/**")</code>	Solo peticiones GET bajo /api/v1/libros/ (el ** cubre sub-recursos).
<code>.permitAll()</code>	Esa regla no exige ninguna autenticación — pasan hasta las peticiones anónimas.
<code>.anyRequest()</code>	La regla "para todo lo demás" — solo lo que ninguna regla anterior haya capturado.
<code>.authenticated()</code>	Exige estar autenticado, sin mirar el rol todavía.
<code>.httpBasic(Customizer.withDefaults())</code>	Activa HTTP Basic — registra el BasicAuthenticationFilter.

El orden importa: gana la primera regla que encaje

⚠ No es la más específica, es la primera en el código

Spring Security evalúa las reglas de `authorizeHttpRequests` de arriba abajo, y aplica la primera que encaje — no la más específica. Si pusieras `.anyRequest().authenticated()` antes que la regla de `/api/v1/libros/**`, esa segunda regla nunca se aplicaría: `anyRequest()` ya lo captura todo. Las reglas concretas van siempre antes que las generales.

Por qué hace falta `.csrf(...disable)`

⚠ Sin esta línea, un POST/PUT/DELETE puede dar 403

Sin esta línea, una petición sin un token CSRF válido devolverá normalmente un 403 Forbidden, aunque incluya credenciales HTTP Basic correctas: el `CsrfFilter` se ejecuta antes que `BasicAuthenticationFilter`.

CSRF protege especialmente a aplicaciones donde el navegador adjunta credenciales automáticamente (cookies de sesión). Aquí se desactiva porque las peticiones añaden explícitamente la cabecera `Authorization` y no usamos cookies — no significa que deba desactivarse en cualquier API REST.

Cerrando la grieta: `AuthenticationEntryPoint`

Prueba a pedir una ruta protegida sin credenciales: el cuerpo del 401 no tiene el formato de tu `ErrorResponse` — es la respuesta por defecto de Spring Security. La razón: ese 401 lo genera un filtro de seguridad, antes de que la petición llegue al `DispatcherServlet` — tu `GlobalExceptionHandler` nunca llega a intervenir.

Spring Security tiene su propia pieza para el mismo trabajo: un `AuthenticationEntryPoint`. Es una interfaz con un único método, `commence(...)`, que se llama exactamente cuando una petición no autenticada intenta entrar en una ruta protegida.

ErrorResponseAuthenticationEntryPoint

```
@Component
@RequiredArgsConstructor
public class ErrorResponseAuthenticationEntryPoint implements AuthenticationEntryPoint {

    private final ObjectMapper jsonMapper;

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
                        AuthenticationException authException) throws IOException {

        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now().toString(), HttpStatus.UNAUTHORIZED.value(),
            "No autenticado", "Se requieren credenciales válidas...", request.getRequestURI());

        response.setStatus(HttpStatus.UNAUTHORIZED.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write(jsonMapper.writeValueAsString(error));
    }
}
```

Por qué aquí no hay ResponseEntity

A diferencia de un `@ExceptionHandler` normal, aquí no hay ningún `ResponseEntity` — trabajas directamente sobre `HttpServletResponse`, porque `commence(...)` se ejecuta antes de que exista ningún mecanismo de Spring MVC que convierta objetos a JSON por ti. Por eso el método usa el `JsonMapper` inyectado y llama a `writeValueAsString(...)` a mano.

No hace falta memorizar la sintaxis exacta: lo que importa es entender que Spring Security tiene su propio punto de entrada para este caso, fuera del circuito normal de tu `GlobalExceptionHandler`.

Registrando el AuthenticationEntryPoint

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http,
    AuthenticationEntryPoint customAuthenticationEntryPoint) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/v1/libros/**").permitAll()
            .anyRequest().authenticated()
        )
        .exceptionHandling(exceptionHandling -> exceptionHandling
            .authenticationEntryPoint(customAuthenticationEntryPoint)
        )
        .csrf(AbstractHttpConfigurer::disable)
        .httpBasic(Customizer.withDefaults())
        .build();
}
```

Por qué en `.exceptionHandling(...)` y no en `.httpBasic(...)`

Podrías registrarlo dentro de `.httpBasic(...)`, y también funcionaría — pero quedaría atado a HTTP Basic. Dentro de dos apartados sustituyes `.httpBasic(...)` por la configuración de JWT, y esa línea desaparecería con él. Puesto en `.exceptionHandling(...)`, sobrevive al cambio sin tocar nada: es independiente de qué mecanismo de autenticación esté activo.

Las limitaciones de esta versión — a propósito

Esta configuración tiene dos problemas serios, y son intencionados: sirven de motivación para lo que viene.

- Usuarios definidos en código y sin persistencia: user y admin se reconstruyen igual en cada arranque. Cualquier modificación se pierde al reiniciar, y cualquiera con acceso al repositorio ve las contraseñas.
- Credenciales que viajan en cada petición: con HTTP Basic, cada petición vuelve a mandar usuario y contraseña, apenas ofuscados en Base64. Más adelante esto se sustituye por JWT.

Ideas clave

- Spring Security, nada más añadirse, cierra todo por defecto — mínima exposición llevada al extremo — con un filtro que examina cada petición antes de llegar al DispatcherServlet.
- Autenticación (¿quién eres?) y autorización (¿puedes hacer esto?) son capas distintas.
- HTTP Basic manda usuario:contraseña codificado en Base64 en la cabecera Authorization — Base64 no es cifrado, solo es seguro sobre HTTPS.
- InMemoryUserDetailsManager es tu UserDetailsService: declara usuarios en código, sin conservar modificaciones entre arranques.

Ideas clave (cont.)

- En `authorizeHttpRequests`, las reglas se evalúan de arriba abajo y gana la primera que encaje — las rutas concretas van antes que `anyRequest()`.
- `.roles("USER")` registra internamente `ROLE_USER`, con el prefijo `ROLE_` añadido por Spring Security.
- Sin `.csrf(...disable)`, un `POST/PUT/DELETE` sin token CSRF da normalmente 403, incluso con credenciales correctas.
- El 401 de Spring Security no pasa por tu `GlobalExceptionHandler` — lo lanza un filtro, antes del `DispatcherServlet`. Un `AuthenticationEntryPoint` a medida lo arregla.